NAQsim 中性原子容错量子计算的全栈架构模拟

面向量子计算理论组的九十分钟技术分享

阅读对象：熟悉量子电路、稳定子和容错基本概念，但不从事量子编译与体系结构研究的同事。

核心问题：一个逻辑量子算法在中性原子容错平台上究竟需要多少时间、多少原子和多大阵列？横向门减少了纠错轮数之后，为什么执行速度仍可能受到原子运动的强烈限制？

核心结论：NAQsim 将逻辑电路、控制资源和阵列几何连接到同一条执行轨迹，再评估时间、空间与逻辑错误率。它发现，小面积表面码 patch 的旋转可能比量子门本身昂贵得多。D3-ROT 是利用这一工具找到的架构优化方案，通过直接旋转、专用旋转 patch 与旋转分散调度共同缓解瓶颈。

本讲义中的实验数字均来自 NAQsim 论文 [1]，按原 PDF 的页码及图表编号定位。标为“教学推导”或“教学示例”的内容用于解释方法，不是论文实验数据。论文版本为 arXiv:2609.14478v1；讲义日期为 2026 年 10 月 13 日。

目录

分享路线与时间分配 3

1 从逻辑资源估计走向真实执行代价 4

2 理解中性原子硬件的五种操作 5

3 从表面码横向门理解旋转约束 6

4 NAQsim 的输入与三阶段结构 8

5 编译阶段将逻辑电路变成可执行的依赖图 9

6 执行阶段从指令展开到激光时间线 11

7 估计阶段连接时间空间与逻辑错误率 13

8 NAQsim 怎样定位 patch 旋转瓶颈 15

9 D3 ROT 为什么需要三个层次一起改变 17

10 实验结果怎样正确阅读 19

11 资源态工厂决定剩下的吞吐瓶颈 21

12 从这项工作提炼理论与架构的共同问题 22

附录 A 讲者页纲与转场提示 23

附录 B 一个完整的纸上执行练习 24

附录 C 常见问答与讨论问题 24

附录 D 复现入口与演示准备 25

参考文献与图表导航 26

分享路线与时间分配

时间	内容	听众应能回答的问题
0–8 分钟	问题定位与资源估计	为什么不能只看逻辑门数或 T-count
8–23 分钟	中性原子硬件和横向表面码	H 后的边界朝向为何会影响后续 CX
23–43 分钟	NAQsim 的三个阶段	如何从电路获得符合硬件约束的时间线
43–55 分钟	误差与空间模型	转移次数如何影响码距与资源预算
55–70 分钟	旋转瓶颈与 D3-ROT	为什么编译策略会随硬件方案改变
70–80 分钟	实验口径与资源工厂	2.27 倍提升意味着什么，有哪些前提
80–90 分钟	理论联系与讨论	哪些问题需要新的纠错或算法研究

附录给出讲者页纲、算例、常见问答和复现入口。主讲可以沿正文推进；详细计算与命令适合会前阅读或会后交流。

1 从逻辑资源估计走向真实执行代价

1.1 门数不是执行时间

理论研究常用逻辑比特数、T-count、T-depth 或查询复杂度衡量算法。它们非常有价值，但不指定硬件执行代价。即使两个电路具有相同的 CX 数和 T 数，只要门依赖关系、比特布局、资源态供应位置不同，其完成时间也可能不同。

中性原子平台尤其明显。一个逻辑 CX 可能需要把两个编码块搬到同一个 patch；一个逻辑 H 的物理门很快，但可能改变编码块的边界朝向；一个 T 门的代价同时涉及资源态、测量和经典条件依赖。逻辑电路中的“一个门”会展开成一系列具有资源占用和空间约束的动作。

教学推导：若第 l 层所有动作的完成时间为 t_end,l，下一层需要等到本层纠错结束，程序时间可写成逐层完成时间的累加。层内动作存在并行时，不能把所有门延迟直接相加。

$$T_{program} = \sum_l T_l,\ T_l = \text{makespan}(\text{层内微操作及 ESM})$$

这里 makespan 指从该层第一项动作开始到最后一项必要动作结束的时间。NAQsim 的重要价值是构造这个时间，而不只是为每种逻辑门分配一个常数。

1.2 NAQsim 回答什么问题

NAQsim 是中性原子容错计算的架构模拟框架。当前论文聚焦采用横向门的表面码架构，输入是 Clifford+T 电路、逻辑指令集、qubit plane 和控制硬件配置，输出包括执行时间、物理比特数、阵列面积及程序成功率估计。[1，p.4–8，Fig.5]

它的“全栈”主要指编译器、控制硬件和量子比特平面的联合建模。这里的架构模拟追踪的是指令、激光动作、原子位置和资源占用；大规模程序的量子态并不在每个时刻逐幅振幅演化。错误率模型则利用 Stim 和解码仿真得到的数据拟合。[1，p.6–8]

因此，研究者可以问：增加 AOD 有多大收益？旋转区需要几个？缩小 patch 会不会损害吞吐？减少搬运次数能否降低所需码距？同一个编译变换在不同硬件上是否仍然有效？这些都是电路统计之外的问题。

1.3 区分三个研究对象

研究对象	主要处理的状态	主要输出
量子态或稳定子仿真	量子态、Pauli 错误、syndrome	概率分布、错误样本、解码结果
量子编译	电路依赖、映射、门与运动安排	符合目标架构的执行计划
NAQsim 架构评估	上述执行计划及硬件时空资源	时间、原子数、面积、LER 估计

NAQsim 使用编译技术完成架构评估，并使用纠错仿真构造误差模型。这三种功能在工具中相互连接，但具有不同的抽象层次。

2 理解中性原子硬件的五种操作

2.1 SLM 保存位置 AOD 实现运动

SLM，即空间光调制器，生成静态光镊阵列。原子通常停在这些陷阱中。AOD，即声光偏转器，生成可移动光镊：先从 SLM 陷阱拾取原子，再改变驱动频率搬运，最后放回目标 SLM 陷阱。行与列的控制具有集体性，因此有些运动可以并行，有些排列变化需要分步完成。[1，p.2，Fig.1]

必须区分 shuttle 与 transfer。Shuttle 是已被 AOD 捕获的原子在空间中运动；transfer 是 AOD 与 SLM 之间的 pick/drop。论文为每次 pick 和 drop 设置了显著时间和噪声代价。由此，距离更短的路线不一定更便宜：如果分段时需要中间 drop 再 pick，固定转移开销会增加；保持 AOD 捕获状态改变方向则不必然增加 transfer。

2.2 Raman Rydberg 和读出

Raman 激光实现单比特操作；Rydberg 相互作用实现两比特 CZ；成像激光用于测量。CX 可通过目标比特上的 H 与 CZ 组合实现。选择性照射允许只操作目标原子对，并允许在原位执行部分门和测量。[1，p.2－3]

传统分区方案使用全局 Rydberg 激光和单独读出区，ESM 需要反复重排和搬运。选择性 Rydberg 与选择性读出可以实现不搬运的 ESM，显著降低纠错轮的时间。注意，“ESM 不搬运”并不表示横向逻辑 CX 和 patch 旋转也不搬运。

2.3 报告使用的参数来自评估设置

操作	论文评估延迟	评估错误率或说明
单比特 H	1 μs	10^{-4}
物理 CZ	1 μs	10^{-3}
Rydberg 寻址 pattern change	1 μs	切换开销
破坏性读出 DRO	60 μs	10^{-3}
非破坏性读出 NDRO	100 μs	10^{-3}
Reset	100 μs	10^{-3}
AOD pick 或 drop	各 40 μs	每次 10^{-3}
AOD shuttle	根据 STA 轨迹估计	T_2 取 10 s
SLM pattern change	1000 μs	更换陷阱图案
SLM turn on/off	1 μs	开关已有图案

来源为 [1，p.8，Table I]。这些是论文仿真配置，不代表所有现有机器的统一性能。正文背景提到更快的物理门演示，但评估应始终使用 Table I，不能把不同出处的最好数字拼在一起。

这一张表已经给出研究动机：如果 H 只有 1 μs，而围绕它的阵列操作涉及多个 40 μs 的 transfer 或 1000 μs 的图案更新，优化物理 H 本身不会消除主要代价。

3 从表面码横向门理解旋转约束

3.1 patch 的三种计数

距离为 d 的 rotated surface code 包含 d^2 个 data qubit 和 d^2-1 个 syndrome ancilla，共 $2d^2$-1 个物理比特。论文的最小 LQ patch 又包含 $(2d+2)^2$ 个 SLM trap，trap 间距为 5 μm，以支持两个 LQ 重叠时仍可独立执行原位 ESM。[1，p.3、5，Fig.2、6]

$$N_{data} = d^2, N_{phys/LQ} = 2d^2 - 1, N_{trap/patch} = (2d + 2)^2$$

原子数、陷阱数、阵列面积是不同指标。空陷阱仍然占几何面积；更大的 patch 可以用同样数量的编码原子，但提供更多运动空间。论文区分物理比特开销与 footprint，正是为了表达这个差别。

术语也要区分：rotated surface code 是一种表面码布局；patch rotation 是运行中把编码块物理旋转。两者中的“旋转”不是同一个概念。

3.2 横向 CX 为什么需要两个 patch 对齐

教学推导：对两个具有对应物理位置与一致 CSS 检查结构的编码块，逐位置执行 CX。Pauli 共轭满足

$$X_c \mapsto X_c X_t, Z_c \mapsto Z_c, X_t \mapsto X_t, Z_t \mapsto Z_c Z_t$$

于是控制块的 X 型稳定子映射为控制与目标对应 X 型稳定子的乘积，目标块的 Z 型稳定子同理。稳定子群保持合适的结构，逻辑 Pauli 获得逻辑 CX 应有的变换。

这一步依赖对应关系。若两块的边界朝向不一致，同一位置的几何配对可能不能保持所需编码结构。论文采用的横向实现要求两个操作数具有相同的 boundary shape。[1，p.3、9，Fig.3、12]

中性原子的实现是把一个 LQ 搬到另一个所在的 patch，使 data qubit 成对相邻，再用选择性激光完成逐对门。一个 patch 可保存一个或两个 LQ；它不等同于“一个逻辑比特永远固定住的格子”。

3.3 横向 H 改变的是编码边界朝向

教学推导：H 将 X 和 Z 互换。把 H 施加到全部 data qubit，会交换表面码的 X 型与 Z 型检查，也交换两类边界的角色。

$$H X H = Z, H Z H = X$$

基线实现把横向 H 与 π/2 物理旋转绑定为 TR_H_ROT，以恢复原边界朝向。这很直观，却比真正的约束更强。真正需要的是下一次横向 CX 的两个输入朝向一致，而不是每个 H 后必须立刻恢复默认朝向。[1，p.9，Fig.12]

为了讲解，定义朝向位 $o \in \{0,1\}$。TR_H 使 o 翻转；ROT 也使边界朝向翻转，但不是额外的逻辑 H。CX 可执行的条件是 $o_c = o_t$。这里只保留了论文需要的两类边界朝向，未表示全部原子坐标。

$$o_i \leftarrow o_i \oplus 1 \text{ (在 TR_H 或 ROT 后)}, o_c = o_t \text{ (在 TR_CX 前)}$$

3.4 一个能看见优化空间的小电路

教学示例：初始 q_0、q_1、q_2 的朝向均为 0。先同时执行 $H(q_0)$、$H(q_1)$，再执行 $CX(q_0,q_1)$，接着执行 $CX(q_1,q_2)$。

步骤	朝向状态	是否需要物理旋转
初始	(0,0,0)	无
$H(q_0)$ 与 $H(q_1)$ 后	(1,1,0)	可先等待
$CX(q_0,q_1)$ 前	(1,1,0)	两操作数一致，不必恢复
$CX(q_1,q_2)$ 前	(1,1,0)	不一致，需要处理
$ROT(q_1)$ 后	(1,0,0)	后一个 CX 可执行

立即恢复默认朝向会做两次旋转；延迟判断只需要一次。这里省略了具体激光调度和每层 ESM，但保留了优化的语义依据。若之后 q_0 又要与 q_2 做 CX，就必须重新检查朝向，不能把此前的省略视为永久正确。

3.5 Pauli frame 与朝向跟踪是两种状态

Pauli frame 记录已知的 Pauli 修正，允许很多修正通过经典更新和测量解释实现。朝向状态记录编码块的几何与检查结构。用两个比特记录 Pauli frame，并不能自动改变需要物理配对的阵列几何。

教学推导：令 frame 为 X^aZ^b，忽略全局相位。H 后 frame 位变为 (b,a)。在 CX 后可按 $a_t \leftarrow a_t \oplus a_c$、$b_c \leftarrow b_c \oplus b_t$ 更新，并保留 a_c、b_t。这个经典更新与前面的边界朝向位是两个独立的信息层。

因此，报告中“省略 Pauli 操作”和“跳过 patch 旋转”应分别解释。前者依赖 frame 跟踪；后者依赖朝向匹配与后续门的要求。

3.6 为什么可以每层只做一轮 ESM

NAQsim 使用横向容错与快速相关解码研究支持的 O(1) ESM 轮数方案，并在每个逻辑指令层末尾对所有仍存活 LQ 执行一轮 ESM。[1，p.4、6–8；延伸阅读 3、4]

横向 CX 的错误传播会使不同编码块之间出现相关性。例如，控制块的 X 错误可传播到目标块；目标块的 Z 错误可传播到控制块。相关解码利用这种跨块传播信息。因此，“每层一轮”依赖门、噪声、检测图与解码方案的一致设计，不能理解为任意稳定子码、任意噪声下都不需要时间冗余。

论文的重点是研究采用这一协议之后的架构代价，并没有在每个应用上重新证明容错阈值，也没有展示完整实时解码器的硬件吞吐评测。

4 NAQsim 的输入与三阶段结构

4.1 四类输入

第一类输入是 OpenQASM 量子程序，论文支持 Clifford+T 门集。第二类输入是逻辑指令集，规定门、准备、资源请求和测量如何表示。第三类输入是 qubit plane，包括 patch 布局、码距、trap 间距和资源态端口。第四类输入是控制硬件，包括激光种类、数量、动作延迟和噪声参数。[1，p.5]

这样的分解允许把同一个算法放到不同架构上评估，也允许固定架构比较不同编译政策。改变码距会同时改变原子数、阵列尺度、运动距离与错误抑制，因而不是只在最后给一个比特数乘法。

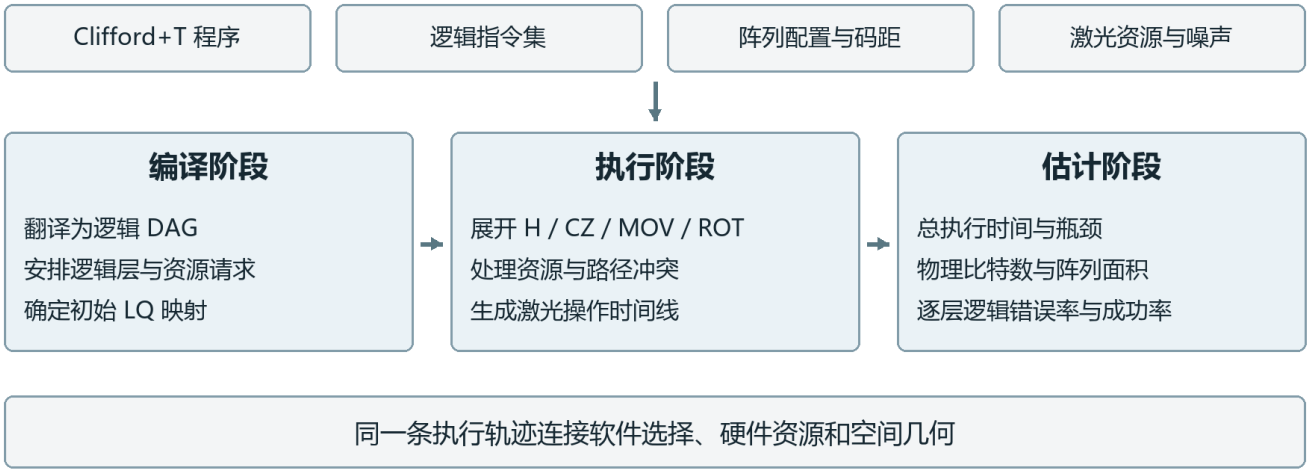


图 1 原创流程示意，依据 [1，Fig.5] 重绘。编译阶段生成逻辑层与初始映射；执行阶段在给定硬件上生成微操作时间线；估计阶段汇总指标。

4.2 为什么分两次安排执行

编译阶段理解容错协议，建立逻辑依赖和层，但不直接绑定某个具体激光的时序。执行阶段才处理“需要几个 AOD”“经过哪些 patch”“这一条路线什么时候会冲突”等细节。[1，p.5–7]

这种分工可以理解为两级调度。第一级回答哪些逻辑操作可以被放在同一层；第二级回答这一层在物理资源限制下怎样执行。第一级允许并行，并不保证第二级能够同时开始。

这一点也是阅读实验结果的关键：层数少、门数少，都不必然使总执行时间更短。层内最长的动作、端口占用与运动冲突同样决定时间。

5 编译阶段将逻辑电路变成可执行的依赖图

5.1 逻辑指令集

指令	含义	需要保留的信息
INIT_X / INIT_Z	准备逻辑 + 态或 0 态	LQ 身份与生命周期
P_X / P_Y / P_Z	可带经典条件的 Pauli 修正	frame 更新及条件来源
TR_H_ROT	基线横向 H 与物理旋转	LQ、朝向与位置
TR_CX	横向逻辑 CX	control、target 及配对要求
M_X / M_Z	逻辑测量	测量基、结果寄存器
M_FB	由先前结果选择 X 或 Z 测量	经典依赖
REQ_Y	请求已准备的 Y 资源态	端口占用与可用性
REQ_TY	同时请求 T 与 Y 资源态	两种端口及新寄存器

来源为 [1，p.5，Fig.6]。优化版本将 TR_H_ROT 拆为 TR_H 与 ROT。指令是逻辑协议的接口，不是一条激光脉冲。

5.2 S 和 T 门为何会引入新依赖

S 和 T 通过资源态与门传送实现。S 使用 Y 态；T 使用 T 与 Y 态及条件测量。论文采用的电路见 [1，Fig.7]。一次传送后，承载程序状态的量子寄存器可能从 Q 变成 Q_t 或 Q_y，翻译器必须跟踪并重命名后续操作数。

教学推导：定义资源态 $|A_\theta\rangle = (|0\rangle + e^{i\theta}|1\rangle) / \sqrt{2}$。对于简单的相位注入电路，数据控制一个指向资源态的 CX，随后测量资源态的 Z。测量分支 0 得到 $\mathrm{diag}(1, e^{i\theta})$，分支 1 得到 $\mathrm{diag}(1, e^{-i\theta})$，差别通过条件修正处理。$\theta = \pi/4$ 时修正包含 S；$\theta = \pi/2$ 时修正是 Z，忽略全局相位。

这个小推导只用于说明非 Clifford 门为何引入资源供应与经典分支；它不是对 Fig.7 的逐线重画。NAQsim 的实际翻译还跟踪状态转移、新寄存器和 M_FB，不能用这个简化电路替代论文协议。

5.3 DAG 是怎样构成的

DAG 即有向无环图。节点表示逻辑指令，边表示操作数依赖。例如，同一 LQ 上两个依次执行的门之间有边；一次测量与读取该结果的条件测量之间也有边。没有路径联系的两个节点才有机会并行，且还要接受资源约束。

这样表示比“第几行代码”更有用，因为前后写出的两个操作可能作用于不同 LQ、互不依赖。反过来，处于同一电路图层的量子门，也可能在门传送展开后因经典反馈与资源态而出现新的依赖。

5.4 ASAP ALAP 与有限端口

普通指令基本采用 ASAP，即依赖满足后尽早安排。REQ 请求采用 ALAP，即在满足后续消费的前提下尽量晚安排，避免提前占据有限资源端口。Pauli 指令通过 frame 跟踪，不在调度中作为物理门执行。[1，p.5 - 6]

端口不是“无穷供应的抽象魔法态”。即使资源态准备好了，只要相关 Y 或 T 端口仍被占用，新的请求也不能安排。传送中的 TR_CX 等操作会释放端口，调度器要跟踪这一状态。过早申请而迟迟不消费，会使端口被锁住，并可能造成调度死锁。

应区分两个问题：端口冲突在编译调度中处理；真实工厂产量不足造成的等待在资源工厂的动态 trace 模拟中另外评估。[1，p.13]

5.5 初始布局与模拟退火

逻辑映射器为程序 LQ 分配初始 patch。工具支持从此前中性原子编译工作迁移来的 simulated annealing，即模拟退火，也支持简单行优先布局。[1，p.6]

模拟退火尝试交换或改变布局，比较运动代价；除接受更好解外，也以随温度下降的概率接受较差解，以避免困在局部极小值。这里最重要的不是具体温度表，而是代价函数是否真正对应硬件的运动方式。

必须注意：论文 artifact 附录明确指出，报告的评估使用 --use-naive-mapping 选择的简单 row-major 布局。不能因为正文介绍了 SA，就把结果归因于 SA 布局优化。[1，p.16]

6 执行阶段从指令展开到激光时间线

6.1 微操作是激光资源层的抽象

一个 micro-operation，简称 μop，是可由同类激光支持的一组物理操作，并可能同时作用于多个 LQ。主要包括 Raman 的 H、成像的 M、Rydberg 的 CZ，以及 AOD 的 MOV 和 ROT。[1，p.6，Fig.8]

例如 TR_CX 需要先使两个 LQ 到达可配对的位置，再执行 H、CZ、H 等物理动作；MOV 又包括 pick、shuttle、drop。ESM 也展开为 H、CZ 与测量序列。一个 LQ 门因此具有多类资源依赖。

激光数量与并行度不是简单的一一对应。一束激光可以覆盖多个目标；AOD 可在没有控制冲突时成组搬运多个 LQ；但另一批目标的寻址模式、几何路径或依赖可能迫使它等待。

6.2 CX placement 是一次匹配问题

当一层有多个 TR_CX 时，需要为每对操作数找到会合 patch。简单地把 control 搬到 target 所在处不总是可行：它们可能已经位于两个不同的 2-LQ patch，目的地没有足够容量。

论文构造二分图：左侧是本层 CX 指令，右侧是可用的候选 patch，排除已是 2-LQ 的 patch。边的权重是把两个操作数搬到该 patch 的代价之和；最小权重全匹配为各 CX 选择不同的会合位置。[1，p.6–7]

教学表达：用 x_jp 表示 CX_j 是否选 patch p。每个 j 恰好选一个 p，每个 p 至多接受一个本层 CX，最小化总代价。

$$\min \sum_{j,p} w_{jp} x_{jp}, \sum_{p} x_{jp} = 1, \sum_{j} x_{jp} \le 1$$

这一优化解决的是候选位置分配，并不等价于整个程序执行时间的全局最优。最终时序还要考虑路径、AOD 数量、旋转与 ESM。

6.3 为什么距离代价出现立方根

在规则 SLM 阵列中，论文允许整块 LQ 沿水平、竖直和对角方向运动。其他方向可能导致移动原子与已有陷阱中的原子碰撞，因而需拆成两个合法方向的步骤。[1，p.7，Fig.9(a)]

论文的 MovingCost 使用 STA 轨迹延迟的距离立方根标度。令 x、y 为起终点坐标差的绝对值，单段合法方向的代价为欧氏距离的立方根；两段运动的代价为两段长度的立方根之和。

$$C_{one} = (x^2 + y^2)^{1/6}, C_{two} = x^{1/3} + y^{1/3}$$

这就是 [1，Eq.(1)] 的等价写法。它是布局与匹配的运动代价，不是带完整单位的 pick/shuttle/drop 总延迟公式。物理时间线仍需加上转移和冲突。

教学示例：若 x=y=8，可走对角单段，代价约为 (128)^(1/6)=2.245；若强制拆成两段，代价为 2+2=4。这里只比较无量纲代价，用来说明“拆段”不能按总长度简单代入一次立方根。

6.4 AOD 执行流水的四个步骤

Decode 把逻辑操作展开并生成目标位置和 AOD μop。Dependency check 检查 LQ 与 patch 依赖并构造 μop DAG。Issue 找到依赖已解除且有 AOD 资源的操作，论文基本优先选择预计最早完成的 μop。Execute 估计 pick、shuttle、drop 的细致时序，处理路线冲突，记录 trace 并更新 LQ 位置。[1，p.6–7]

例如，两个 LQ 重叠时要分别旋转，可能先需要搬走一个；某个 patch 的原有 LQ 离开之后，另一个移动才能进入。这些都是操作的前后关系。AOD 不足则是结构资源冲突。两条路线在中途同时经过同一 patch，则是进一步的时空冲突。

6.5 route conflict 必须看时间区间

两条运动即使没有数据依赖，也不能在同一时刻穿过同一个 patch。若简单地让后一条运动等待前一条全部完成，会过度估计时间。论文对每条路线经过的 patch 记录进入与退出时间，只补上避免同时占用所需的延迟。[1，p.7，Fig.9(c)]

教学示例：运动 A 总共持续 200 μs，在中间 patch P 的占用区间为 [60,100] μs。运动 B 原计划在 [80,120] μs 经过 P。将 B 的开始时间后移 20 μs，就使它在 [100,140] μs 占用 P，不必等待 A 的 200 μs 全程结束。

$$\Delta t = \max(0, t_{\text{exit},A} - t_{\text{enter},B})$$

这个公式用于“A 先通行、且单个冲突区间”的教学情况。完整模拟要对所有沿途 patch 迭代检查；一次后移可能在其他地方产生新冲突，还要结合 AOD 及操作依赖。

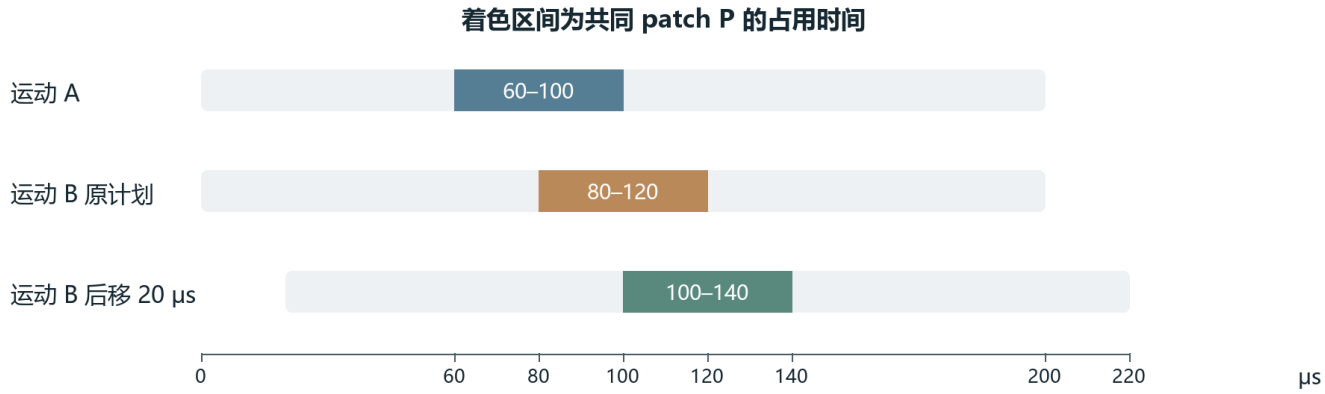


图 2 教学时间线，使用上面的自定数值。阴影区段只表示共同经过 patch P 的时间，不代表完整运动时间。

6.6 每层完成后对所有存活 LQ 纠错

NAQsim 在每层末尾执行一轮 ESM，对象是所有尚未被破坏性测量的 LQ。[1，p.6] 因而一个没有执行逻辑门的活跃 LQ，也可能为等待中的这一层贡献 memory LER。

这使“为了争取并行而延长某一层”具有潜在误差代价。调度影响的不是只有被移动的比特；长时间等待和更多活跃层会改变全程序错误预算。

7 估计阶段连接时间空间与逻辑错误率

7.1 时间与空间从同一执行计划得到

执行时间取自微操作 trace 的完成时间。物理比特数按照实际配置的 LQ patch 数与 $2d^2-1$ 计算；footprint 是量子比特平面面积，用来表示空陷阱、运动空隙和较大旋转区带来的额外空间。[1，p.7]

不能直接用“输入程序逻辑比特数乘 $2d^2-1$”替代整个架构计数：实际平面可能包含布局空位、端口和额外的操作空间。工厂的额外空间也应按其配置另计。Table II 的架构比特数应直接读取，或由完整配置复算，而不是仅从程序宽度猜测。

做理论资源估计时，至少保留四个量：程序逻辑宽度、配置 patch 数、物理比特数、footprint。它们分别回答算法占多少逻辑工作寄存器、架构要预留多少位置、用了多少编码原子、设备需要多大几何范围。

7.2 transfer 噪声是一个关键新增维度

一次 MOV 包含 pick 和 drop。论文在每次 AOD–SLM transfer 后，对 LQ 的物理比特施加 depolarizing noise。[1，p.7] 这解释了为什么减少 pick/drop 不仅提高速度，也可能改善逻辑错误率。

为避免混淆，下面 n 指一层内 transfer 的计数。一次完整“拾取后放回”包含两次 transfer，而不是把它当作 n=1 的一个无噪声搬运动作。旋转方案中的多次拾取与放下同样进入计数。

这样的误差抽象适合高效架构探索，但把真实转移误差简化成 Pauli 噪声。泄漏、原子丢失、集体强度噪声、位置相关相关性等效应，需要额外模型才能用于更具体的硬件判断。

7.3 单块与双块使用不同 LER 模型

论文将一层的操作分为单 LQ 操作和横向 CX 双 LQ 操作。单块使用单轮 memory 操作的 LER 近似；双块结合相关解码仿真，显式区分 control 与 target 的 transfer 计数。[1，p.7–8，Eq.(2)、(3)]

$$P_{L,1Q}(d,n) = c_0(c_1 + c_2 n)^{d/2}$$

$$P_{L,2Q}(d,n_c,n_t) = c_3(c_4 + c_5 n_c + c_6 n_t)^{d/2}$$

系数	目标噪声设置的拟合值	所在模型
c_0	0.040	单 LQ 前因子
c_1	0.036	单 LQ 基础项
c_2	0.009	单 LQ transfer 项
c_3	0.036	双 LQ 前因子
c_4	0.056	双 LQ 基础项
c_5	0.003	control transfer 项
c_6	0.012	target transfer 项

这些数值来自 [1，Fig.10(c) 的行 (b)]。它们对应 Table I 参数，不能移植到不同噪声条件，也不能与放大噪声验证时的行 (a) 混用。d/2 是拟合形式；不要把它改成自己更熟悉的 (d+1)/2 后继续使用相同系数。

control 和 target 系数不同提示：在这个已拟合的协议里，搬哪一块可能影响错误预算。它不证明所有门、所有 decoder 都具有同样的不对称性；拓展到新的设置仍须重新验证。

7.4 为什么大程序需要代理模型

若每个候选架构都直接抽样到 10^{-12} 级别的逻辑错误率，成本会极高。论文用 Stim 与 PyMatching 生成可测量的 LER 数据，再拟合解析模型，供大量架构配置快速查询。[1，p.8]

验证方法值得完整讲述。论文用 d=3 至 13 的数据拟合，再用 d=23 的数据检查预测。为使 d=23 有足够错误样本，验证时将物理错误参数放大：单块 11 倍，双块 7 倍。$\log_{10}$ LER 的 RMSE 分别为 0.162 和 0.097。然后在 Table I 的目标噪声设置下另行拟合目标系数。[1，p.8，Fig.10]

这支持模型形式的预测能力，但不是对目标参数下 d=23 极低 LER 的直接高精度抽样验证，也不是任何程序及噪声条件下的严格误差界。报告中应把“模型得到验证”与“所有低错误率数字均直接测量”分开。

7.5 全程序成功率怎样理解

NAQsim 逐层累计所有存活 LQ 的 LER，得到全程序成功率估计。[1，p.7] 单轮 LER 很小并不够，因为大程序有极多的逻辑块与层。

教学推导：如果将不同错误事件视为独立，成功率近似为各项 (1-p_i) 的乘积；在 p_i 很小时，负对数成功率近似为错误率之和。即使不假设独立，union bound 也给出总失败概率不超过各事件概率之和，但前提是 p_i 的事件定义和边际概率正确。

$$P_{\mathrm{succ}} \approx \prod_i (1 - p_i), \; -\ln P_{\mathrm{succ}} \approx \sum_i p_i$$

这里是帮助理解“累积”意义的推导，不声称正文完整指定了跨层独立概率模型。双块 CX 的相关性已在其 LER 建模中处理，也不能再把这个相关事件随意拆成两个独立单块事件重复计数。

7.6 减少搬运可能改变码距

教学算例：只考虑 $M=10^9$ 个同分布单 LQ 事件，错误预算 $\varepsilon=10^{-3}$，不考虑 CX、工厂与其他误差。用目标系数要求 $M \cdot P_L \leq \varepsilon$，可得到

$$d \geq 2 \ln(\varepsilon/(Mc_0)) \,/\, \ln(c_1 + c_2 n)$$

当 n=0 时，最小奇数码距约为 15；当每个事件 n=4 时，约为 19。单块物理比特数由 449 增为 721，增加约 61%。固定 d=15 时，n 从 0 到 4 会使该拟合模型的 LER 增加 $2^{15/2}$，约 181 倍。

这个例子不是论文 workload 的预测，只展示“调度改变 transfer → LER 改变 → 所需码距改变 → 空间与运动距离改变”的反馈关系。真实程序需要逐层计算、区分不同操作，再验证整个预算。

8 NAQsim 怎样定位 patch 旋转瓶颈

8.1 为什么反射旋转很慢

在最紧凑的 patch 内，不能随意让所有原子同时沿圆弧移动。论文基线通过水平与对角反射组合实现 π/2 旋转，反射又由逐行交换原子完成。随着 d 增大，需要顺序处理更多行，因此基线反射旋转具有 O(d) 的延迟开销。[1，p.9、11，Fig.11]

这是一项几何与控制代价，和物理 H 脉冲的时间几乎没有关系。只优化 H 门保真度或给 H 再缩短几百纳秒，不会解决多轮 atom pick/drop 的耗时。

8.2 关键路径上的旋转时间

论文将 AOD 时间分为 Move、Route conflict 和 exposed Rotation 等部分。Exposed rotation 是在实际调度中未被其他动作掩盖、最终增加墙钟时间的旋转开销，不能把每个 LQ 的旋转时间简单相加当作程序时间。[1，p.9、11]

在原始基线的 workload 分析中，旋转占总执行时间的 55.3%–77.6%，平均 61.9%。这是论文用于架构优化的主要瓶颈。[1，p.2、9、Fig.11(a)、16]

教学推导：若其他部分保持不变，只把占比 f 的瓶颈加速 s 倍，则 Amdahl 关系为

$$S = 1 / [(1 - f) + f/s]$$

以 f=0.619 为例，即使理想地消除旋转，单独改变这一项的上限约为 1/0.381=2.62 倍。这解释了为何处理一个占比很高的动作就可能带来两倍级收益。实际 D3-ROT 也改变运动与冲突等部分，因此这里不能作为其速度的精确预测。

8.3 先放宽不必要的语义约束

编译优化把 TR_H_ROT 拆成 TR_H 与 ROT，再沿 CX 子图分析边界朝向。遇到不匹配时，给旋转朝向的操作数插入 ROT，并更新后继 CX 的朝向信息。[1，p.9–10]

这不是把所有 H 后旋转全部删除，而是把强制立即执行变为只在匹配约束需要时执行。论文使用 BFS 遍历与传播实现一个启发式策略，没有声称求出全程序最少旋转次数的全局最优解。

ROT 与 H 解耦还创造合法调度窗口。窗口由相关的前后 CX 等依赖约束确定，可以跨越 H 所在位置。对于某个 patch，物理旋转可能安排在相关 H 之前或之后，只要逻辑门语义与后续横向配对条件保持正确。[1，Fig.12(c)]

8.4 集中与分散是两种不同目标

若反射旋转可对多块集体执行，一层里旋转一个或很多 LQ 的主要延迟可能相近。那么应把 ROT 集中到尽量少的层，减少“含旋转的层数”。论文的 aggregation 先统计各层可容纳的 ROT 数量，再贪心选择数量最多的层。

若直接旋转只有少量专用 patch，则一层允许的旋转数量有上限。这时应分散 ROT，避免瞬时需求过大。论文的 distribution 根据合法窗口安排到较早的尚未使用层，降低峰值旋转数。[1，p.10]

编译政策	旋转总数相对基线	含旋转层比例	时间相对基线	每层最大旋转数的平均值
Baseline	100%	35.9%	100%	17
Skip	89.8%	34.7%	97.1%	16.9
Skip 加 Aggregation	89.8%	15.6%	64.7%	55.5
Skip 加 Distribution	89.8%	43.8%	109.2%	1.6

来源为 [1，p.10，Table III 的 Average 列]。最后一列是“各 workload 的每层最大值”的平均，不是全部层的平均旋转数。Distribution 使各 workload 的峰值降至 1–3，但在原反射硬件上反而平均慢 9.2%。不能把减少峰值本身等同于速度提升。

这也是理论组可记住的一条联系：优化目标不是一个抽象电路图的天然属性，而是图与执行资源共同决定的。

9 D3 ROT 为什么需要三个层次一起改变

9.1 两种直接替代方案仍有明显代价

时间高效的反射可以并行交换更多原子，但需要至少 2.25 倍 footprint，并增加 pick/drop 从而提高 LER。直接旋转仍受原有静态 SLM 光阱限制，需临时移除相关光阱；若在运行时修改 SLM 图案，约 1 ms 的开销又会抵消收益。若所有 patch 都预留直接旋转所需空间，footprint 还会约翻倍。[1，p.10，Fig.13、15]

因此，仅提出“换一种更快的旋转”并不足够。必须同时解决目标原子的路径、非目标原子的保存、阵列占地与旋转请求的并发需求。

9.2 第一个 D 是直接旋转

控制硬件方案采用 SLM toggling。先用 AOD 捕获需要保存的原子，在直接旋转区域暂时关闭 SLM 陷阱，完成目标 patch 的旋转，再打开 SLM 并放下。开关现有 SLM 图案比重新计算和加载图案更快，论文将开关时间设为 1 μs。[1，p.11，Fig.14(a)、(b)]

这个操作仍然需要光镊保持原子，也仍有 pick/drop 噪声。在全平面采用 DIR_TOGL 的中间方案中，平均 exposed rotation 为 12.3 μs，比 SLM pattern-change 方案减少 25.1 倍；但它需要约 2 倍 footprint，且增加非目标原子的 transfer。

12.3 μs 是中间方案 DIR_TOGL 的旋转指标，不能把它写成最终 D3-ROT 的平均旋转时间。

9.3 第二个 D 是专用旋转 patch

qubit plane 改为异构布局：大部分是最小 normal patch，只有少量 rotation patch 加大到可直接旋转的面积。normal 与 rotation 区域使用不同 SLM 激光，目标 LQ 先搬到专用区域，再旋转。[1，p.11，Fig.14(c)]

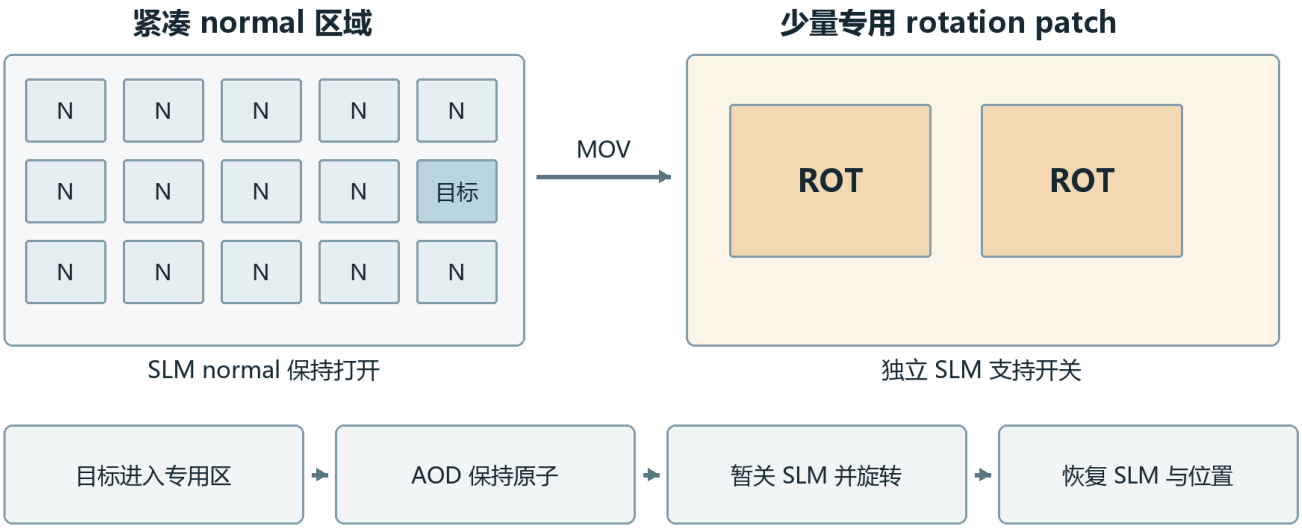


图 3 原创机制示意，依据 [1，Fig.14]。图示布局与块数用于教学，不是论文的实际坐标文件。

新增面积只集中在少量位置，代价是前往旋转区的 MOV、排队和潜在 route conflict。论文估计 footprint 时，把包含较大 patch 的整行或整列的尺度保守扩大 √2，因而面积开销不能仅按“三个 patch 翻倍”除以 patch 总数计算。[1，p.11]

9.4 第三个 D 是分散旋转

专用旋转区的数量限制一层能并行旋转的 LQ 数。此时，先前让原反射基线变慢的 distribution 政策变成合适的调度：把允许移动时刻的 ROT 分散到不同层，避免需要大量旋转区。

在论文八个 workload 上，经过 Skip+Distribution，每层最多需要三次旋转，因此三块专用旋转 patch 足以满足这些 benchmark 的需求。[1，p.10–11，Table III] 这是该 workload 集的结果，不是任意 Clifford+T 电路只需三块旋转区的定理。

9.5 协同机制比单项技巧更重要

直接旋转降低一次旋转的物理时间；专用 patch 避免全平面面积翻倍；分散调度限制对这些 patch 的峰值需求。三项互相补足，形成 D3-ROT。

若只添加专用 patch 却沿用 aggregation，可能在某一层需要几十个 LQ 同时旋转，反而排长队。若只采用 distribution 却保留慢反射，更多层都付出旋转时间。若只采用全平面 SLM toggling，又承担面积和转移噪声代价。这些机制在论文中通过不同配置与消融展示，而不是通过“旋转门数减少”一个指标解释。[1，p.10–11，Fig.15、16]

10 实验结果怎样正确阅读

10.1 八个 workload 覆盖了哪些计算

workload	任务	程序 LQ 数	Transversal 码距	LS 码距
Hei	二维 Heisenberg QPE	167	23	27
FH	二维 Fermi－Hubbard QPE	95	23	27
Jellium	电子气 QPE	41	23	27
H4	小分子 QPE	27	17	21
adder	可逆加法	64	13	17
qft	量子 Fourier 变换	29	15	19
ising1d	一维 Ising 时间演化	64	15	19
ising2d	二维 Ising 时间演化	64	17	21

来源为 [1，p.8，Table II]。Transversal 与 LS 采用各自满足 99.9% 程序成功率目标的码距，比较没有固定同一个 d。

QPE 部分模拟 qubitization 的 SELECT 子程序，再按目标能量精度 0.01 所需的次数放大成本。四个重复次数分别为 48067、56549、38075、975。Table II 给出的门数对应被评估的电路单元，解释总程序规模时必须结合重复次数。[1，p.8]

其他电路来自 FTCircuitBench，经 Qiskit 将任意角旋转综合为 Clifford+T，程序级综合误差上限为 10^{-3}。这个综合精度指标与硬件随机失败率预算的物理含义不同，不应在未指定误差度量时直接相加。

10.2 编译优化与最终架构收益

只做旋转 skip，平均删除 10.2% 的旋转，但平均时间只减少 2.9%。原因是许多层仍然至少包含一次昂贵旋转。加入 aggregation 后，平均时间减少 35.3%，对应论文约 1.55 倍加速。[1，Table III、Fig.16]

最终 D3-ROT 相对原始 Baseline 的平均速度提升为 2.27 倍，最大为 2.57 倍，出现在 Hei；平均 footprint 增加 22.4%。这些是八个 benchmark 在既定参数下的架构仿真结果。[1，p.11，Fig.16]

旋转实现比较 Fig.15 还给出：exposed rotation 从 520 μs 降为 D3-ROT 的 49.8 μs；D3-ROT 的平均层延迟为 842.3 μs，理想零开销直接旋转方案为 740.1 μs，两者相差 13.8%。[1，p.11，Fig.15]

Fig.15 的层均值与 Fig.16 的原始程序基线消融属于不同统计比较。不要把 520 μs 除以 61.9% 来倒推某个“统一 Baseline 总层时间”，也不要把平均速度比解释成任一 workload 都恰好加速 2.27 倍。

10.3 与 lattice surgery 的比较

论文通过 Quration 模拟 LS。每个程序 LQ 被分配 2×2 patch 区域，额外 patch 用于 surgery 和 code deformation；NA 的 ESM 取 237 μs，超导取 1 μs。LS 的 T 门 reaction time 取 d 轮 ESM。[1，p.12]

架构	执行时间相对超导 LS	主要原因
超导 LS	1	1 μs ESM 基准
中性原子 LS	237	相同 LS 轮数配 237 μs ESM
中性原子横向 Baseline	58	轮数少，但运动与旋转昂贵
中性原子 D3-ROT	25.5	大幅缓解旋转瓶颈

来源为 [1，Fig.17 与 p.12]。在该模型中 D3-ROT 仍慢于超导 LS，但较少的比特空间使空间时间成本差距更小：Baseline 与 D3-ROT 分别约为超导 LS 的 9.0 倍和 3.9 倍。[1，Fig.4、17]

这些跨平台比值依赖各平台的 ESM、码距、协议、布局和错误设置。它们可解释该研究条件下的趋势，不能用来宣布任何中性原子机器都慢 25.5 倍，或任何实际超导设备均达到这个理论基线。

还要区分 physical-qubit time 与 footprint time。专用旋转区增大几何面积，而不必同比增加编码原子数；评估光学系统可用面积时，不能把二者混成一个 space-time 指标。

10.4 AOD 资源敏感性

Pick/drop 延迟	AOD pairs	D3-ROT 时间	Baseline 时间
40 μs	充足	1.00	2.27
40 μs	8	1.03	2.55
40 μs	4	1.09	3.52
120 μs	充足	1.27	3.49
120 μs	8	1.36	4.11
120 μs	4	1.45	5.81

来源为 [1，Fig.18]。图中报告跨 workload 的几何平均延迟，全部时间都归一到默认 D3-ROT，即 40 μs、充足 AOD；不是每种架构各自归一为 1。

在最保守一列条件下，D3-ROT 比自身默认配置慢 45%，而 Baseline 比自己的默认配置慢约 5.81/2.27=2.56 倍；此时 D3-ROT 相对 Baseline 的加速约为 5.81/1.45=4.01 倍。这说明所选旋转方案更少依赖大量 transfer，但不表示 AOD 数量与速度从此不重要。

11 资源态工厂决定剩下的吞吐瓶颈

11.1 快的逻辑数据路径需要快的资源供应

S 和 T 指令将资源态消费纳入数据路径。在主程序逻辑执行加快后，资源态工厂可能来不及供应。REQ 的端口可用性与工厂产量是两回事，必须同时检查。

论文针对 QPE workload 额外评估 resource-state factory。采用 magic state cultivation 为 15-to-1 distillation 提供较低误差的 T 态，并计算 Y-state distillation 成本。Y 态是稳定子资源态，T 态是非稳定子 magic state；不要把二者都描述为非 Clifford magic state。[1，p.13，Table IV]

教学联系：理想 15-to-1 distillation 的主导输出误差约为 $35p^3$。若输入 $p=3\times10^{-6}$，则该主导项约为 9.45×10^{-16}。但真实输出还受工厂内部逻辑错误和协议实现影响，不能只凭这个数宣称整个工厂达到了同样精度。

11.2 用 trace 驱动供应与等待

论文对每种工厂数量配置运行动态 trace 模拟。每个静态逻辑层开始前检查资源是否足够；不足时等待概率性工厂运行产生资源态，累计由产量不足导致的延迟。扫描工厂数量就得到时间与空间的 Pareto frontier。[1，p.13，Fig.19]

教学推导：若某类工厂每次成功概率为 s，批次时间为 τ，有 F 个独立工厂，则长期平均产率约为 Fs/τ。这个平均值只能作为初步判断。突发需求、端口容量、缓存、不同类型资源的联合请求和失败随机性仍会造成等待，因此必须看实际 trace。

$$\lambda_{\text{supply}} \approx Fs / \tau$$

如果只比较平均 T-consumption 与平均产率，可能漏掉某几层的集中消费。旋转 distribution 的思想与此相似：峰值需求和平均需求是不同的优化对象。

11.3 工厂加入后需要重新比较

在工厂配置扫描中，论文 Fig.19 标出的最佳时空成本点，相对免费且及时提供资源态的理想基准，使 D3-ROT 与 LS 的空间×时间成本分别成为 1.81 倍和 1.14 倍。D3-ROT 的额外负担更明显，其中包括较快逻辑执行相对于 MSC 产量的压力，以及横向 MSD 每批需要 15 个 distilled Y 态。[1，p.13]

这些是 QPE 分析的工厂成本乘子，不是八个 workload 都增加同样时长。主结果 2.27 倍与 Fig.17 的比较不能直接说成“已包含所有工厂、解码控制与真实实验开销的端到端性能”。

对理论算法研究，这意味着 T-count 仍然重要，但更完整的问题是：何时消费 T 与 Y？每层消费峰值多大？算法和综合策略能否使工厂持续工作而减少缓存与等待？

12 从这项工作提炼理论与架构的共同问题

12.1 朝向是一个可以进入优化的离散变量

TR_H/ROT 翻转朝向，CX 添加两个变量相等的约束。这可以形成新的编译问题：在因果约束和旋转资源约束下，怎样选择翻转次数与时机，使程序完成时间或面积时间最小？

论文给出了 BFS 与贪心调度启发式。理论上可以研究优化问题的复杂度、特定电路族的结构，以及基于整数规划或约束求解的较优基准。一个更小的旋转门数解，也可能因资源聚集而具有更长 makespan，目标函数需明确。

12.2 错误传播图与物理执行图必须相容

相关解码利用逻辑算法中的跨块错误传播，而运动实现改变错误位置与转移次数。若允许 patch 暂时保持不同朝向，或者改变 CX 的会合位置，实际 syndrome extraction 与 detector graph 是否还与错误模型一致？

这应通过电路与检测事件的明确建模回答。编译结果保持理想 unitary 语义，并不自动证明带噪实现保持相同容错性质。NAQsim 提供轨迹与代价基础，而新的协议仍需相应的纠错验证。

12.3 码距应和调度一起选择

较少 transfer 可能允许较小 d，较小 d 又可能减小运动距离和所需空间。可以把目标写为：在程序失败率不超过 ε 的条件下，联合选择码距、布局、旋转区数、工厂数与调度，使时间或空间时间最小。

论文按 workload 选择达到目标成功率的 d，展示了跨方案比较的必要性。进一步的联合搜索、多码距混合和位置相关码距，是适合后续研究的方向，不能当作本论文已实现的完整功能。

12.4 低距离或大规模相关噪声下需要重新校准

紧凑模型用 transfer 次数解释 LER，适合搜索大量方案。如果真实设备出现跨原子的共同噪声、丢失、泄漏或旋转阶段相关错误，相同 n 的两种轨迹可能具有不同风险。

值得研究的不是只多加一个“保守误差率”，而是寻找仍可快速查询的特征：方向、驻留时间、密度、操作重叠、原子损失可检测性和跨 patch 相关范围，并验证代理模型对这些特征的响应。

12.5 qLDPC 的联系应明确为扩展方向

论文讨论了 surface-code 计算加 qLDPC 存储的 load-store 架构，也指出指令翻译、调度、映射，以及 μop 分解和调度等通用模块可经适当修改扩展到 qLDPC。[1, p.13–14]

当前结果仍是横向表面码，不能声称已经完成一般 qLDPC 架构评测。要扩展，需要重新定义逻辑操作、空间布局、检查电路、错误传播与 LER 模型；“模块化框架可扩展”与“某种新码已验证”是两个不同阶段。

12.6 与 NISQ 中性原子编译的关系

NISQ 编译通常直接管理物理比特、Rydberg 门与运动；这里管理的是编码块，还必须处理 ESM、边界朝向、状态传送、资源端口、逻辑错误预算和长期程序成功率。

NAQsim 借用了此前 ZAC 的映射、CX placement 和相关思路，并针对 FTQC 的受限移动方向及 route conflict 扩展。[1, p.14] 因此熟悉 PowerMove、Atomique 等工作有助于理解运动优化，但不能直接照搬物理电路的成本函数。FTQC 的 H、S、T 也不能当成零代价或统一常数。

附录 A 讲者页纲与转场提示

页纲	讲述重点	建议图或公式
1 问题	从算法资源到墙钟时间	三个同门数但不同路径的场景
2 工具定位	NAQsim 的输入输出	Fig.5 对应的本讲义图 1
3 硬件词汇	SLM AOD Raman Rydberg 成像	论文 Fig.1
4 代价尺度	门快 transfer 慢	Table I 参数表
5 编码块	data ancilla trap footprint	三种计数公式
6 横向 CX	对应位置与朝向约束	CSS 共轭公式
7 横向 H	X Z 与边界互换	H 共轭与朝向位
8 小电路	两个 H 可以省一次旋转	三比特朝向表
9 容错前提	每层一轮 ESM 的条件	跨块错误传播示意
10 翻译	S T 资源态与寄存器变更	论文 Fig.7
11 逻辑调度	DAG ASAP ALAP 端口	一次资源请求与消费
12 微操作	逻辑并行不等于物理并行	论文 Fig.8
13 运动分配	二分图匹配与代价	matching 约束与 Eq.(1)
14 时空冲突	中途占用而非全程串行	本讲义图 2
15 错误模型	transfer 与相关解码	Eq.(2)、(3)
16 码距反馈	少搬运可以省空间	d=15 与 d=19 教学例
17 瓶颈发现	旋转为何占比高	Fig.11 与 Amdahl 关系
18 编译政策	集中和分散的适用场景	Table III 摘要
19 三项协同	三个 D 如何互相补足	本讲义图 3
20 数据读法	消融与平台比较口径	Fig.16、17
21 工厂	更快主程序造成供给压力	Fig.19 与产率公式
22 讨论	朝向优化 误差模型 联合搜索	附录 C 的问题

开场可说：“我们平常估计 T-count 与逻辑比特数，今天再问这些门在一块有几何结构、有有限激光资源的编码阵列上要怎样执行。”

进入 NAQsim 时可说：“接下来把逻辑并行、激光并行和空间通行三个不同层次放在一起，看看同一段电路怎样得到真实时间线。”

进入 D3-ROT 时可说：“删掉不必要的旋转只是第一步。真正的瓶颈仍来自一次旋转的物理实现，所以我们需要改变旋转的地点和执行条件。”

进入讨论时可说：“这项工作把算法与硬件之间几项容易隐藏的变量显式化了。下一步可以问，哪些变量可以由更好的理论协议消除，哪些变量需要架构设计承担。”

附录 B 一个完整的纸上执行练习

这是教学练习，不是 NAQsim 的实测或运行日志。

第一步，选取正文三比特电路，画出 H_0、H_1、CX_{01}、CX_{12} 的依赖图，给出初始朝向全为 0。对比“每个 H 后立即 ROT”和“只在 CX 前检查朝向”的两种展开，前者两次 ROT，后者一次。

第二步，给 LQ 设初始位置 P_0、P_1、P_2。令 CX_{01} 需要把 q_0 搬到 P_1；CX_{12} 前 q_1 还需恢复朝向。讨论在 P_1 已含两个 LQ 时能否直接旋转 q_1，以及是否需要先搬出其中一个。这一步体现逻辑朝向优化之后仍存在物理容量约束。

第三步，把某次 MOV 拆成 pick、shuttle、drop，按 Table I 为 pick 与 drop 各分配 40 μs，再加入符号化 shuttle 时间 τ_s。单次无冲突 MOV 的时间为 $80\ \mu s+\tau_s$，并有两次 transfer。若中间放下再拾取，就新增固定转移开销；若只保持捕获状态改变运动方向，则必须按该具体实现重新判断 transfer 数，不能仅凭路径有拐弯推断。

第四步，安排另一条运动也经过共同 patch P。采用正文 [60,100] 与 [80,120] μs 区间，只把后者推迟 20 μs。然后再检查 AOD 是否够用，以及新路线是否碰到另一项动作。

第五步，在层结束加入所有 live LQ 的 ESM。使用单块与双块的对应模型估计每个事件的 LER，保留事件身份以避免双重累计。最终列出：层数、ROT 总数、最大并发 ROT、MOV 次数、transfer 次数、激光完成时间、原子数、footprint 和错误预算。

这个练习的目标不是手工复刻所有 simulator 细节，而是建立判断习惯：语义正确、依赖合法、几何合法、资源可用、误差预算满足，这些条件须同时成立。

附录 C 常见问答与讨论问题

为什么不把所有 H 都移到电路末尾

H 与其他门通常不对易。把 H 穿过 CX 需要改变门的角色或电路表示，还可能影响测量与反馈。论文主要移动物理 ROT 的执行时机，不是无条件移动逻辑 H。

两个朝向都错了为什么还能做 CX

“默认朝向”只是参考。对于论文的实现约束，两块具有相同的边界形状和对应关系，就可执行横向 CX。真正的问题是彼此不一致，而不是是否处于某个被指定的绝对方向。

near rotation free 是否意味着零旋转

没有。ROT 仍会执行，目标也仍可能搬到专用区域。Near-rotation-free 描述的是主要暴露旋转开销被大幅缓解；最终平均 exposed rotation 仍为 49.8 μs。[1，Fig.15]

增加很多 AOD 为什么不能直接解决问题

更多 AOD 只能解除部分资源冲突。紧凑 patch 内顺序反射、SLM 图案切换与几何限制不能单靠增加 AOD 数量完全消失。论文默认充足 AOD 来展示这种瓶颈，并另做有限 AOD 的敏感性分析。

最值得理论组讨论的五个问题

1. 能否为特定 Clifford 电路族建立朝向翻转的最优或近似算法，并将旋转资源约束一起纳入？
2. 若保持非默认朝向更长时间，实际 syndrome 电路与相关 decoder 需要怎样同步变更？
3. 减少 transfer 得到的 LER 收益能否稳定转化为较小码距，是否存在反向的路径变长或拥堵效应？
4. 对不同综合策略，T 与 Y 的消费峰值和缓存需求怎样变化，算法层能否主动平滑资源态需求？
5. 当引入丢失与空间相关噪声后，用 transfer 次数作主要特征的代理模型还能在哪些范围内保持可靠？

附录 D 复现入口与演示准备

论文公开仓库为 https://github.com/naqsim/naqsim 。截至讲义编写时，可见仓库页面主要包含 README 与 LICENSE；可复现实验包及详细流程见论文 artifact appendix 的 Zenodo 链接。以下命令依据本地论文附录，不表示已在本机执行。[1，p.15 – 16]

论文列出经过 artifact evaluation 的归档：https://zenodo.org/records/21542759 。其后续版本补充敏感性分析 raw data，DOI 为 https://doi.org/10.5281/zenodo.22684078 。获取和解压归档后，应先阅读包内 README 并核对目录及版本。

推荐演示顺序是先复绘已有数据，再跑一个 workload 的端到端模拟。复绘只检验数据分析与绘图，不重新运行 simulator；完整模拟还要执行逻辑编译与硬件执行阶段。

```
# 在解压后的 NAQsim 根目录准备论文测试环境
uv sync
# 切到 artifact evaluation 目录
cd src/tsc_naa_sim/artifact_evaluation
# 从归档 raw_data 重建图和表
./run_replotting.sh
# H4 是较轻的 QPE 端到端示例
# Fig.17 和 Fig.19 需先构建外部 Quration 并生成相应数据
./quration/run.sh --workload h4 \
  --quration-bin /absolute/path/to/quration/build/main/qret
./run_full_simulation.sh h4 --num-threads 4
```

上述路径与 wrapper 来自 [1，p.16]，适用于 Linux/macOS shell 或合适的 WSL 环境，不是 Windows PowerShell 命令。论文测试环境为 Python 3.13.3、uv 0.8.11；完整流程依赖 Quration，特别是 Fig.17、19。完整实验约需 30 GB 磁盘空间，在论文服务器上约需 30 小时，不能把“一分钟复绘”说成“一分钟重跑论文”。

现场分享适合提前完成复绘，把 Fig.11、Table III、Fig.16、Fig.18、Fig.19 的生成图准备好。若要展示端到端能力，提前跑 H4 并保留输入、配置、trace 和汇总表；会上解释一条逻辑指令怎样变成 μop，再展示相应延迟分解。

参考文献与图表导航

[1] Y. Ueno et al. NAQsim Full-Stack Architecture Simulation Framework for Fast and Space-Efficient Neutral Atom Quantum Computing. arXiv:2609.14478v1, 2026. 主文与 artifact appendix 均以工作目录中的 NAQsim.pdf 为准。公开入口：https://arxiv.org/abs/2609.14478 。

[2] S. Sunami, A. Goban, H. Yamasaki. Transversal surface-code game powered by neutral atoms. arXiv:2506.18979, 2025. 本文 [1] 的横向表面码与门传送协议来源。https://arxiv.org/abs/2506.18979 。

[3] M. Cain et al. Correlated Decoding of Logical Algorithms with Transversal Gates. Physical Review Letters 133, 240602, 2024. https://doi.org/10.1103/PhysRevLett.133.240602 。

[4] M. Cain et al. Fast correlated decoding of transversal logical algorithms. arXiv:2505.13587, 2025. 本文 [1] 采用的快速相关解码与低 ESM 轮数建模基础。https://arxiv.org/abs/2505.13587 。

[5] W.-H. Lin, D. B. Tan, J. Cong. Reuse-Aware Compilation for Zoned Quantum Architectures Based on Neutral Atoms. HPCA 2025, pp.127–142. 本文 [1] 的 ZAC 技术来源，工作目录有同题论文。

[6] J. Ruan et al. PowerMove Optimizing Compilation for Neutral Atom Quantum Computers with Zoned Architecture. ASPLOS 2025, pp.163–178. https://doi.org/10.1145/3676642.3736128 。用于进一步理解 NISQ 运动优化背景。

原论文位置	本讲义用途
p.2–4 Fig.1–4	硬件、表面码操作、研究动机
p.4–7 Fig.5–9	架构流程、指令、调度、AOD 冲突
p.7–8 Fig.10 Eq.(1)–(3)	距离代价与 LER 拟合
p.8 Table I、II	参数、benchmark 与码距
p.9–10 Fig.11–13 Table III	旋转瓶颈、skip、聚合与分散
p.11 Fig.14–16	三项架构协同与消融
p.12 Fig.17、18	LS 比较与 AOD 敏感性
p.13 Table IV Fig.19、20	工厂开销与 H S 消融
p.13–14	qLDPC 与此前编译工作的关系
p.15–16	artifact 版本、环境与命令